S10 — Mission Intelligence Simulator

Architecture, 4-Member Ownership & 5-Day Implementation Plan

1. Project Definition

S10 is the Mission Intelligence layer of the ANSA Reality Engine. Its defined role is to turn natural-language mission intent into executable flight plans and dynamically replan those plans when conditions change. It supports multi-objective optimization (fuel vs. time vs. risk) and validates autonomous decisions before they reach real hardware.

The catalogue defines S10 as PLANNED, Layer 6 — Mission & AI, difficulty 8/10, owned by DERYK + AI + Robotics. Required behaviors: natural-language-to-mission conversion, dynamic in-flight replanning, multi-objective route optimization, and autonomous decision validation. Outputs: mission graphs, waypoint sequences, FSM events and risk scores. □filecite□turn3file0□

2. Inputs and Outputs

Input	S10 Processing	Output
Mission intent	Interpret intent and build mission representation	Mission graph
Navigation information	Route/constraint processing	Waypoint sequence
Weather/environment conditions	Constraint and risk evaluation	Updated plan / risk score
Communication conditions	Communication-aware planning/replanning	FSM events / revised plan
Current mission state	Dynamic replanning	Updated mission graph + waypoints

3. Proposed Architecture

The catalogue specifies capabilities and dependencies, not a source-code layout. The following is a proposed modular design based directly on those requirements.

```
s10_mission_intelligence/
├── include/
│   ├── s10_mission.h
│   ├── s10_graph.h
│   ├── s10_waypoint.h
│   ├── s10_constraints.h
│   ├── s10_risk.h
│   ├── s10_fsm.h
│   └── s10_interface.h
├── src/
│   ├── core/         mission_manager.*, mission_state.*, mission_context.*
│   ├── intent/       intent_parser.*, mission_builder.*, command_interpreter.*
│   ├── planning/     route_planner.*, waypoint_generator.*, graph_planner.*, replanner.*
│   ├── optimization/ objective_manager.*, cost_model.*, route_optimizer.*
│   ├── validation/   decision_validator.*, constraint_checker.*, safety_checker.*
│   ├── risk/         risk_engine.*, risk_estimator.*
│   ├── fsm/          mission_fsm.*
│   ├── adapters/     sitl_adapter.*, navigation_adapter.*, weather_adapter.*, communication_adapter.*
│   ├── output/       mission_graph_writer.*, waypoint_writer.*, event_writer.*
│   └── logger/       mission_logger.*
├── config/           mission.yaml, objectives.yaml, scenarios.yaml
├── tests/
├── docs/
├── logs/
└── README.md
```

4. Core Data Flow

```
Mission Intent
   ↓
Intent Parser → Mission Builder → Mission Graph
   ↓
Constraint / Environment Analysis
   ↓
Route + Waypoint Planner
   ↓
Multi-Objective Optimizer
   ↓
Decision / Safety Validation
   ↓
Executable Mission Plan
   ↓
Mission State Updates
   ↓
Replanner → Updated Mission Graph / Waypoints / FSM Events / Risk Score
```

5. Four-Member Ownership

Member	Area	Main Files	Responsibility
Member 1 – bindu	Mission Core + Interfaces	mission_manager.*, mission_state.*, mission_context.*, shared headers	Mission data model, lifecycle, public interfaces and integration.
Member 2 - aya	Intent + Mission Construction	intent_parser.*, mission_builder.*, command_interpreter.*	Convert mission intent into structured mission graph.
Member 3 - aya	Planning + Optimization	route_planner.*, waypoint_generator.*, graph_planner.*, replanner.*, objective_manager.*, cost_model.*, route_optimizer.*	Routes, waypoints, fuel/time/risk optimization and replanning.
Member 4 - bindu	Validation + Risk + FSM + Outputs	decision_validator.*, constraint_checker.*, safety_checker.*, risk_engine.*, mission_fsm.*, output/*, logger/*	Decision validation, constraints, risk scores, FSM events and defined outputs.

6. Five-Day Plan

DAY 1 — Architecture & Contracts

- Member 1: mission data model, state/context, interfaces and project skeleton.
- Member 2: intent schema, command representation and mission graph schema.
- Member 3: route/waypoint/objective/cost interfaces and planner skeleton.
- Member 4: constraint/risk/FSM/output schemas.
- Team: shared contracts frozen and project builds.

DAY 2 — Mission Creation + Basic Planning

- Member 1: mission manager and state lifecycle.
- Member 2: basic intent parser + mission graph builder.
- Member 3: basic route planner + waypoint generator.
- Member 4: constraint checker + initial risk/FSM logic.
- Team: intent → mission graph → basic waypoint plan → validation.

DAY 3 — Optimization + Validation

- Member 1: integrate planning/validation pipeline.
- Member 2: support multiple mission task types and graph validation.
- Member 3: multi-objective fuel/time/risk optimization.
- Member 4: decision validation, safety checks, risk score and output writers.
- Team: complete first usable S10 plan.

DAY 4 — Dynamic Replanning + Adapters

- Member 1: live mission-state update path and integration.
- Member 2: graph modification/rebuild on changed conditions.
- Member 3: dynamic replanning and route replacement.
- Member 4: S01/S04/S12/S11 adapter inputs into constraints, risk and FSM.
- Team: changed condition → updated mission plan.

DAY 5 — Integration + Validation

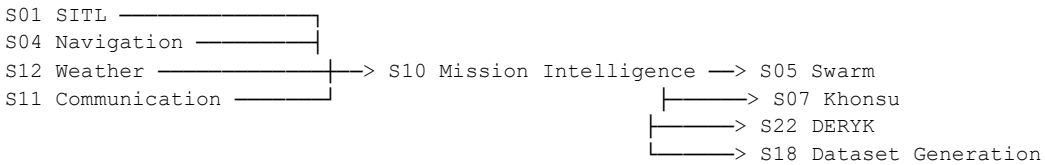
- Member 1: final integration, API freeze and documentation.
- Member 2: validate intent conversion and mission graphs.
- Member 3: validate optimization and replanning scenarios.
- Member 4: validate risk/FSM/output/logging.
- Team: demo, tests, sample missions, docs and integration-ready release.

7. File Dependencies

Module	Depends On	Reason
mission_manager.*	Member 2 + Member 3 + Member 4	Coordinates mission creation, planning and validation.
mission_builder.*	s10_mission.h + graph structures	Creates the common mission representation.
route_planner.*	Mission graph + navigation adapter	Needs tasks and navigation constraints.
route_optimizer.*	Route planner + objective/cost model	Evaluates candidate routes.
replanner.*	Mission state + route planner + current constraints	Replans the active mission after changes.
decision_validator.*	Constraint checker + safety checker + risk engine	Validates whether a plan is acceptable.
mission_fsm.*	Mission state + validated plan	Generates mission-state events.
output/*	Mission graph + waypoints + FSM + risk	Produces the catalogue-defined S10 outputs.

8. External Simulator Dependencies

The S10 profile explicitly lists S01 SITL, S04 Navigation, S12 Weather and S11 Communication as dependencies. S10 should consume their information rather than duplicate their responsibilities. □filecite□turn3file0□



9. Broader Integration Notes

The catalogue separately shows S13 Power & Battery being consumed by S10 and S19 Digital Earth being consumed by S10. These are broader platform integrations; they are not listed as direct S10 dependencies in the S10 profile, so they should be treated as additional inputs/extensions rather than silently added to the core dependency chain. □filecite□turn3file4□

The catalogue's dependency graph places S10 downstream of its foundation and upstream of S22, while S18 also consumes S10. □filecite□turn6file0□

10. Five-Day MVP Scope

- Convert mission intent into a structured mission graph.
- Generate waypoint sequences.
- Optimize candidate plans using fuel/time/risk objectives.
- Validate autonomous decisions and constraints.
- Generate risk scores and FSM events.
- Trigger dynamic replanning when mission conditions change.
- Provide modular interfaces for S01, S04, S12 and S11.
- Produce mission graphs, waypoint sequences, FSM events and risk scores.

11. What S10 Should NOT Duplicate

- Do not build a second weather simulator; consume S12.
- Do not build a second communication simulator; consume S11.
- Do not replace S04 Navigation.
- Do not implement cybersecurity; that is S15.
- Do not make S10 the swarm coordinator; S05 is the downstream swarm/fleet module.

12. Acceptance Criteria

- Mission intent becomes a valid mission graph.
- Waypoint sequence is generated.
- Fuel/time/risk objectives are configurable.
- Unsafe or infeasible plans are rejected or flagged.
- Risk score is generated.
- FSM events are generated.
- Changed conditions can trigger replanning.
- Updated plan can replace the active plan.
- Outputs/logs are reproducible.
- External simulator interfaces remain modular.

13. Final Architecture Principle

S10 should be a reusable mission-planning substrate, not a drone-specific script. The core operates on generic mission objects, constraints, objectives, routes and states. Simulator-specific data enters through adapters, while standardized outputs are provided to S05, S07, S18 and S22.

S10 = Mission Intent + Planning + Optimization + Validation + Dynamic Replanning + Mission Outputs